

PORTADA

Análisis y Diseño de Sistemas II

Semana 9 — SOLID: Interface Segregation Principle (ISP)

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**¿Por qué un robot que no come
tiene que implementar el método `comer()` ?**

TRANSICIÓN

La cuarta letra de SOLID

Ya vimos **SRP** (una clase, una razón para cambiar) y **OCP/LSP** (extender sin romper, sustituir sin sorpresas). Hoy vemos la **I de SOLID: Interface Segregation Principle (ISP)** — el mismo problema de responsabilidades mezcladas que vimos en clases, pero aplicado a interfaces.

Es el primero de los dos principios que cierran la **Unidad 4** esta semana: el viernes vemos el último, **Dependency Inversion**, junto con el laboratorio de cierre de unidad.

CONTRA EJEMPLO

La interfaz "gorda" Trabajador

```
interface Trabajador:
    def trabajar(self): ...
    def comer(self): ...

class Empleado implements Trabajador:
    def trabajar(self): print("Empleado trabajando")
    def comer(self): print("Empleado almorzando")

class RobotTrabajador implements Trabajador:
    def trabajar(self): print("Robot trabajando")
    def comer(self):
        raise NotImplementedError("un robot no come")
```

`RobotTrabajador` está obligado a implementar `comer()` solo porque la interfaz `Trabajador` lo declaró — aunque nunca lo va a usar de forma válida.

CONCEPTO

El problema: interfaces gordas (fat interfaces)

Una **interfaz gorda** ("fat interface") es una interfaz que agrupa métodos que pertenecen a **responsabilidades distintas**, pensada para servir a varios tipos de cliente a la vez en lugar de a uno solo.

- `Trabajador` mezcla la responsabilidad de "trabajar" (aplica a cualquier trabajador) con la de "comer" (solo aplica a trabajadores biológicos).
- Cualquier clase que implemente `Trabajador` queda forzada a declarar **todos** sus métodos, los use o no.
- Las clases que no pueden cumplir un método terminan lanzando excepciones, dejándolo vacío, o retornando valores sin sentido — todas son señales de una interfaz mal segmentada.

CONCEPTO

Definición formal de ISP

Interface Segregation Principle (Robert C. Martin): **los clientes no deben verse forzados a depender de métodos que no usan.**

Aquí "cliente" es cualquier clase que depende de una interfaz para usarla — no el usuario final del sistema. Si una clase cliente solo necesita uno de los métodos de una interfaz, no debería tener que conocer ni implementar los demás por el simple hecho de compartir la misma interfaz con otro cliente que sí los necesita.

CONCEPTO

La solución: dividir por rol, no por clase

En lugar de una interfaz única, se define una interfaz pequeña **por cada rol o capacidad** que un cliente realmente necesita.

Cada clase implementa solo las interfaces que le aplican.

```
interface Laborable:
    def trabajar(self): ...

interface Alimentable:
    def comer(self): ...

class Empleado implements Laborable, Alimentable:
    def trabajar(self): print("Empleado trabajando")
    def comer(self): print("Empleado almorzando")

class RobotTrabajador implements Laborable:
    def trabajar(self): print("Robot trabajando")
```

CONCEPTO

Qué se gana al segregar

- `RobotTrabajador` ya **no implementa** `comer()` — desaparece el método vacío o la excepción artificial.
- Un cliente que solo necesita "algo que trabaje" depende de `Laborable`, no de `Trabajador` completa — no conoce métodos que no le interesan.
- Agregar un rol nuevo (por ejemplo `Descansable`, con `dormir()`) no obliga a tocar `Laborable` ni `Alimentable` — cada interfaz sigue siendo pequeña y estable.

CONCEPTO · RELACIÓN CON LO VISTO

ISP es SRP aplicado a interfaces

SRP (semana 8) dice que una clase debe tener una sola razón para cambiar. **ISP dice exactamente lo mismo, pero sobre interfaces:** una interfaz debería agrupar solo los métodos que cambian juntos, por la misma razón, para el mismo tipo de cliente.

`Trabajador` violaba SRP tanto como ISP: mezclaba dos razones para cambiar (la lógica de trabajo y la lógica de alimentación) en un solo contrato. Separarla en `Laborable` y `Alimentable` resuelve ambos principios a la vez, porque son la misma idea vista desde dos ángulos.

CONCEPTO · RELACIÓN CON LO VISTO

ISP y la cohesión de la semana 7

En la semana 7 definimos **cohesión** como qué tan relacionados están los elementos dentro de un mismo módulo, y vimos que baja cohesión es señal de un **God Object** agrupando responsabilidades que no deberían estar juntas.

Una interfaz gorda es el mismo síntoma, un nivel más arriba: en vez de una clase con demasiadas responsabilidades, es un **contrato** con demasiadas responsabilidades. Una interfaz cohesionada agrupa solo métodos que un mismo tipo de cliente necesita en conjunto — igual que una clase cohesionada agrupa solo atributos y métodos relacionados entre sí.

TENSIÓN CONCEPTUAL

ISP no significa "una interfaz por método"

Un error común es leer ISP como licencia para **fragmentar en exceso**: crear una interfaz distinta para cada método individual "por si algún cliente solo necesita ese uno". Eso multiplica el número de tipos que hay que conocer y navegar, sin ninguna ganancia real de diseño.

La segregación correcta agrupa por rol coherente, no por método aislado: `Laborable` agrupa lo que todo trabajador necesita para trabajar, aunque en el futuro tenga más de un método. El criterio es el mismo de la cohesión — juntar lo que cambia junto, separar lo que no.



INTERACCIÓN

¿Interfaz gorda o interfaz correcta?

En el chat, para cada interfaz digan si viola ISP y, si lo hace, cómo la dividirían:

1. `ImpresoraMultifuncion` con `imprimir()` , `escanear()` y `enviarFax()` , implementada también por una impresora que solo imprime.
2. `Repositorio<T>` con `guardar()` y `buscarPorId()` , implementada por todos los repositorios del sistema que necesitan ambas operaciones.
3. `Vehiculo` con `conducir()` y `volar()` , implementada tanto por `Carro` como por `Avion` .

5 minutos · respondan en el chat

CONCEPTO

Señales de que una interfaz viola ISP

Señal	Qué implica
Método vacío o con <code>pass</code>	La clase implementa el método solo para cumplir el contrato, sin comportamiento real
Excepción tipo <code>NotImplementedError</code>	La clase declara que ese método de la interfaz nunca debería llamarse en su caso
Parámetros o retornos ignorados	El método existe pero no usa lo que la interfaz le exige recibir o devolver
La interfaz crece con cada cliente nuevo	Cada vez que aparece un tipo de cliente distinto, se le agregan métodos a la misma interfaz en vez de crear una nueva

CONCEPTO · LO QUE VIENE

Interfaces pequeñas, la base del principio del viernes

Dividir Trabajador en Laborable y Alimentable no solo resuelve ISP: también deja el terreno listo para el último principio SOLID, **Dependency Inversion**, que vemos el viernes.

DIP dice que el código de alto nivel debe depender de **abstracciones** —como estas interfaces pequeñas— en vez de depender directamente de clases concretas. Interfaces bien segregadas, como las de hoy, son exactamente el tipo de abstracción que DIP necesita para funcionar.

SÍNTESIS

Ideas clave de hoy

1. **ISP**: los clientes no deben verse forzados a depender de métodos que no usan.
2. Una **interfaz gorda** mezcla responsabilidades de distintos tipos de cliente en un solo contrato.
3. La solución es dividir en interfaces pequeñas, una por rol o capacidad, no una por método.
4. **ISP es SRP aplicado a interfaces**: una interfaz, una razón para cambiar.
5. Se conecta con la **cohesión** de la semana 7 — agrupar lo que pertenece junto, separar lo que no.
6. Señales de violación: métodos vacíos, excepciones de "no implementado", parámetros ignorados.



CIERRE

Para el viernes

Revisen las interfaces de su diagrama de clases del proyecto: identifiquen si alguna mezcla métodos que distintos tipos de cliente usan de forma diferente. Tráiganla identificada, junto con una propuesta de cómo la dividirían.

El viernes cerramos la Unidad 4 con **Dependency Inversion (DIP)**, el último principio SOLID, y el laboratorio de cierre de unidad donde cada equipo aplica los cinco principios completos sobre su propio proyecto — insumo directo del **Momento 1 (20%)** que se evalúa el miércoles 30/09.

